

Dosya Organizasyonu I: Sıralı ve Doğrudan Dosyalar

CEN207 Veri Yapıları — Hafta 13

Dr. Öğr. Üyesi Uğur CORUH · 2026-2027 Güz

Bugünün planı (3 saat)

Saat	Konu
1	Bloklar, kayıt/alanlar Anim 1 · bloklama çarpanı Anim 2 · sıralı arama Anim 3
2	Sıralı dosyada ikili arama Anim 4 · sıralı güncelleme/birleştirme Anim 5
3	Görelî erişim Anim 6 · kovalara hash'leme Anim 7 · ilerleyici taşıma Anim 8 · mezar taşları Anim 9

Öğrenme çıktıları: LO.1 (temel veri yapılarını açıklama) · LO.2 (karmaşıklık analizi) · LO.6 (C ve Java'da doğru uygulama) · LO.7 (doğru yapıyı seçme)

Bu haftanın kavramları — nerede

Kavram	Nerede
Kayıt/alanlar, bloklama çarpanı	Bölüm 2–3
Sıralı arama, sıralı dosyada ikili arama	Bölüm 4–5
Sıralı güncelleme (ana + işlem birleştirme)	Bölüm 6
Görelî (doğrudan) erişim	Bölüm 7
Kovalara hash'leme, ilerleyici taşıma, mezar taşları	Bölüm 8–10

Kod örnekleri nasıl çalışır

- Her fikrin eksiksiz bir **C** ve **Java** programı var
- İkisi de gerçek dosya G/Ç yapar: `fopen` / `fseek` / `fread` / `fwrite`, `RandomAccessFile`
- Her program kendi **geçici lab klasörünü** oluşturur, sonra siler
- Kaynaklar: `code/week-13/c/` ve `code/week-13/java/`
- Her programın beklenen çıktısı hafta notlarında var

Hatırlatma — hesaplanan adresler (Hafta 1)

- $\text{arr}[i] = \text{base_address} + i * \text{element_size}$
- i indeksine ulaşmak için **arama** gerekmez — yalnız aritmetik
- Bölüm 7'nin görelî dosyası tam olarak bu fikrin diske taşınmış hali
- $\text{block} = \text{rrn} / \text{bf}$, $\text{offset} = \text{rrn} \% \text{bf}$ — aynı hesaplanan-adres fikri

Hatırlatma — hash'leme ve çakışmalar (Hafta 6)

- $h(\text{key}) = \text{key} \bmod m$, RAM'de bir hücre seçer
- Çakışmalar **zincirleme** ya da **açık adresleme** ile çözülür
- Bölüm 8–10, her iki fikri de mantığında değişmeden yeniden kullanır
- Tek fark: her "hücre" artık bir disk konumu

Gerçekten yeni olan tek fikir

- Hafta 1–12: maliyet = **karşılaştırmalar**, takaslar, işaretçi atlamaları
- Bu hafta: maliyet = **blok okuma ve yazmalar**
- Bir blok okuma, herhangi bir RAM işleminden kat kat pahalıdır
- Zaten RAM'deki fazladan kayıtları karşılaştırmak neredeyse bedavadır

Haftanın haritası — genel bakış

Aile	Bölümler
Temeller	Kayıt/alanlar (2), bloklama çarpanı (3)
Sıralı — sırayla işle	Arama (4), sıralı arama (5), güncelleme (6)
Doğrudan — konumu hesapla	Görelî (7), hash'leme (8–10)

1. Dosyalar Neden Farklıdır: Blok Yönelimli G/Ç

Başlangıç sorusu

Bir dosya genellikle RAM'e hiç sığmaz. Bir program 20 baytlık bir kayıt istiyorsa, disk neden yalnız 20 baytı vermez?

Fikir: bir blok, G/Ç'nin en küçük birimidir

- Diskler sabit boyutlu **bloklara** organize edilir
- Her okuma/yazma **tam bir blok** aktarır, asla daha azını
- 20 baytlık bir istek bile çevresindeki tüm bloğu yükler
- Dosya organizasyonunun bir konu olarak var olma nedeni tam olarak bu

Bu haftanın çizim kuralı

- Bir **disk** satırı: adlı, numaralı bloklar, içerikleri içlerinde çizili
- Bir **RAM tamponu** satırı: şu an yüklü tek blok
- Sağda sayılan **blok okumaları** ve **blok yazmaları**
- Bu sayı, bu haftanın $O(\dots)$ 'sudur — anahtarlara değil, ona bakın

İki aile, bir hafta

Aile	Bir kaydı bulma
Sıralı (bölüm 4–6)	Blokları sırayla oku, ya da ortadaki bloğa atla
Doğrudan (bölüm 7–10)	Bir hesaplama, sonra bir blok okuma

Sıralı, **her** kaydı zaten işleyecekseniz kazanır; doğrudan, milyonlarca kayıt arasından **tek** bir kayıt için kazanır.

Mini soru

Sıralı bir arama 100 blok okuyor, blok başına 1 anahtar karşılaştırıyor. İkinci bir dosya 100 blok okuyor, blok başına 1000 anahtar karşılaştırıyor. Hangisi daha hızlı?

Yanıt

Hiçbiri — neredeyse aynı maliyet. Disk erişimi baskındır; zaten yüklü bir bloktaki fazladan anahtarları karşılaştırmak neredeyse bedavadır.

2. Kayıt ve Alanlar

Başlangıç sorusu

Bir ad alanı "Al" ya da "Christopherson" tutabilir. Her kayıt, gerçek uzunluğundan bağımsız olarak aynı sayıda bayt mı ayırmalı?

Aynı alan için üç yerleşim

Yerleşim	Nasıl	Maliyet
Sabit uzunluk	<code>NAME_FIXED</code> 'e doldur ya da kırp	Boşa giden yer ya da kayıp veri
Sınırlandırılmış	Değeri yaz, sonra bir sınırlayıcı bayt	Sınırlayıcıyı bulmak için tarama
Uzunluk önekli	Uzunluk baytı yaz, sonra o kadar bayt	$O(1)$ okuma, en çok 255 uzunluk

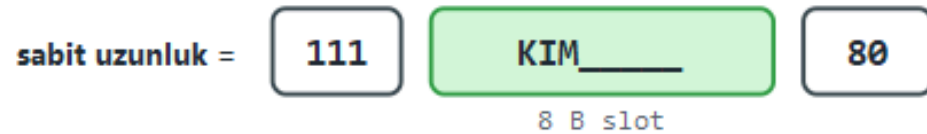
Sabit uzunluk neden en çok önemlidir

- Her kayıt **tam olarak aynı boyutta**
- Bir konumdan adres hesaplamayı mümkün kılan tam olarak bu
- Bölüm 3 (bloklama) ve bölüm 7 (doğrudan erişim) ikisi de bunu ister
- Sınırlandırılmış/uzunluk önekli kayıtlar bu şekilde bulunamaz

Benzetme — bir formdaki "ad" kutusu

- Bir kağıt formun ad kutusu sayfada **sabit** boyutta
- Bir e-tablo hücresi ne yazarsanız yazın **büyür** — sınırlandırılmış
- Bir kargo etiketi önce bir uzunluk basar, sonra o kadar karakter
- Üçü de gerçek sistemlerde var, tam olarak bu takaslar için

Kayıt ve alanlar, adım adım



total = 176 B (wasted 55 B)



total = 132 B



total = 132 B

0 kayıt kırıldı, sabitte toplam 55 bayt boşa gitti.

Bitti: 11 kayıt. Sabit = 176 B (boşa giden 55 B, 0 kırpma), sınırlayıcı = 132 B, uzunluk önekli = 132 B. Sabit uzunluk daha basit okunur ama yer israf eder ya da veri kaybeder; ikisi de aynı toplam bilgiyi taşır.

Uç durum — 8 karakterden uzun her ad



total = 160 B (wasted 0 B)



total = 190 B

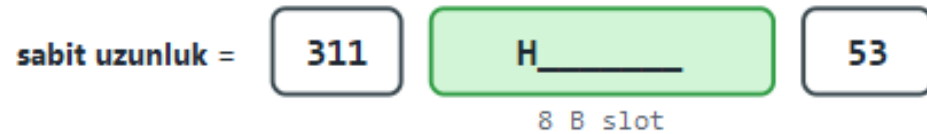


total = 190 B

10 kayıt kırıldı, sabitte toplam 0 bayt boşa gitti.

Bitti: 10 kayıt. Sabit = 160 B (boşa giden 0 B, 10 kırpma), sınırlayıcı = 190 B, uzunluk önekli = 190 B. Sabit uzunluk daha basit okunur ama yer israf eder ya da veri kaybeder; ikisi de aynı toplam bilgiyi taşır.

Uç durum — boş bir ad ve çok uzun bir ad



total = 176 B (wasted 50 B)



total = 147 B



total = 147 B

4 kayıt kırpıldı, sabitte toplam 50 bayt boşa gitti.

Bitti: 11 kayıt. Sabit = 176 B (boşa giden 50 B, 4 kırpma), sınırlayıcı = 147 B, uzunluk önekli = 147 B. Sabit uzunluk daha basit okunur ama yer israf eder ya da veri kaybeder; ikisi de aynı toplam bilgiyi taşır.

Kod — sabit uzunluk: doldur ya da kırp

```
#define NAME_FIXED 8

int n = (int) strlen(name);
if (n >= NAME_FIXED) {
    /* too long: truncate, data is LOST */
    memcpy(r.name, name, NAME_FIXED);
} else {
    memcpy(r.name, name, n);
    /* pad with filler bytes */
    memset(r.name + n, '_', NAME_FIXED - n);
}
```

Kod — uzunluk önekli: geri okuması $O(1)$

```
unsigned char len =  
    (unsigned char) strlen(name);  
fwrite(&len, 1, 1, fp);    /* 1-byte prefix */  
fwrite(name, 1, len, fp);  
  
/* reading back: read len, then read  
   exactly len more bytes – no scanning */
```

Karmaşıklık

- Bir alanı yazmak/okumak: her üç yerleşimde de **$O(\text{alan uzunluğu})$**
- Fark **asimptotik değil**
- Boşa giden yer (sabit) ile ayrıştırma maliyeti (sınırlandırılmış) ile hiçbirisi (uzunluk önekli, 255 sınırıyla) arasında

Yaygın hatalar

- Tam ya da kırpılmış sabit bir alanın `'\0'` için yer olmadığını unutmak
- Verinin içinde yasal olarak görünebilecek bir sınırlayıcı seçmek
- Veri kaybını bildirmek yerine **sessizce** kırpma

Mini soru

Uzunluk önekli bir alanın okuma maliyeti neden $O(1)$ artı kendi uzunluğu iken, sınırlandırılmış bir alanınki tam bir tarama?

Yanıt

Uzunluk önekli okuyucuya boyut önceden söylenir (bir bayt, sonra o kadar bayt).

Sınırlandırılmış bir okuyucu, sonu bulmak için bayt bayt sınırlayıcıyla karşılaştırmalıdır.

3. Bloklama Çarpanı

Başlangıç sorusu

Bir blok 100 bayt; bir kayıt 20 bayt. Disk her okumada 80 bayt mı israf ediyor, yoksa birkaç kayıt bir bloğu paylaşabilir mi?

Fikir: bloğa `bf` kayıt sığdırmak

- `bf = floor(blockSize / recSize)`
- Kayıtlar küçük bir **RAM tamponunu** birer birer doldurur
- Tampon `bf` kayda ulaştığı an: **tek** bir `fwrite`
- Disk erişimi **kayıt başına değil, blok başına** olur

İki tür israf

Tür	Nerede	Neden
İç parçalanma	Her tam blok	<code>bf * recSize</code> , <code>blockSize</code> 'a nadiren tam eşit
Son blok israfı	Yalnız son blok	Kayıt sayısı <code>bf</code> 'in tam katı değil

Benzetme — bisikletli kurye değil, nakliye kamyonu

- Bisikletli kurye seferde **bir** paket taşır: yavaş, tek tek
- Nakliye kamyonu seferde **bf** kutu taşır: aynı kutu, az sefer
- Yarı boş çıkan bir kamyon yine de tam sefer yakıtı yakar
- O "yarı boş sefer" tam olarak bu bölümün son blok israfı

Bloklama çarpanı, adım adım



Bitti: 13 anahtar → 3 blok yazıldı. Bloklama çarpanı $bf = 5$ olmasaydı (kayıt başına 1 disk erişimi), 13 erişim gerekirdi; şimdi yalnız 3. İç israf toplam 0 B, son blok israfı 40 B.

Uç durum — bloktan büyük kayıt

HATA: kayıt (60 B) bloktan (50 B) büyük — hiçbir kayıt sığmaz!

disk =

blocks written = 0

internal waste = 0 B

last-block waste = 0 B

RAM tampon =

bf = 0 — bu bir tasarım hatası, blok boyutu büyütülmeli ya da kayıt küçültülmeli. Hiçbir yazma yapılmaz.

Uç durum — 12 anahtar $bf=3$ 'e tam bölünür



Bitti: 12 anahtar → 4 blok yazıldı. Bloklama çarpanı $bf = 3$ olmasaydı (kayıt başına 1 disk erişimi), 12 erişim gerekirdi; şimdi yalnız 4. İç israf toplam 40 B, son blok israfı 0 B.

Kod — kapasite, doldurma ve flush

```
#define BLOCK_SIZE 100

int block_capacity(int rec_size) {
    return BLOCK_SIZE / rec_size;
}

/* block_put: append a key; flush when full */
/* block_flush: write a PARTIAL last block */
/*   anyway – a whole block, however few keys */
```

Karmaşıklık

- Bloklamanın kendisi hiçbir algoritmanın **asimptotik** maliyetini değiştirmez
- **Sabiti** değiştirir: kayıt sayısı değil, blok okuma/yazma sayısı
- Daha büyük **bf** → daha az, daha büyük disk erişimi → pratikte daha hızlı
- Bir blok RAM'de rahatça tamponlanamayacak kadar büyüyene kadar

Yaygın hatalar

- **Son bloğun** yine de tam bir blok yazma maliyeti taşıdığını unutmak
- `blockSize` 'ı bir kayıttan küçük seçmek (`bf = 0`)
- İç parçalanmayı son blok israfıyla karıştırmak — ikisi bağımsızdır

Mini soru

`recSize = 24`, `blockSize = 100`. `bf`
nedir, ve **tam bir blok** kaç bayt iç
parçalanma israf eder?

Yanıt

$bf = 4 \cdot 4 \cdot 24 = 96$ bayt kullanıldı,
tam blok başına **4 bayt** israf — burada
küçük görünür, ama koca bir dosyadaki her
blokla çarpılır.

4. Bir Dosyada Sıralı Arama

Başlangıç sorusu

Bir dosyanın kayıtları sıralı değil. Bir anahtarı bulmak için, anahtar çıkana ya da dosya bitene kadar her bloğu okumaktan başka bir seçenek var mı?

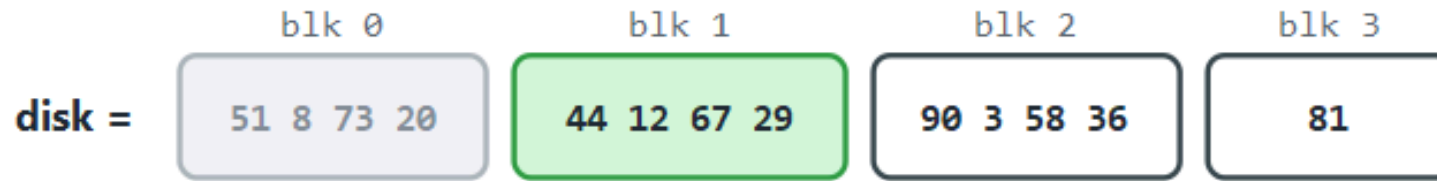
Fikir: blokları sırayla oku, içeride karşılaştır

- Blok 0'ı RAM tamponuna oku
- İçindeki **her** kaydı hedefle karşılaştır
- Bulunamadıysa: blok 1'i oku — ve böyle devam et
- Önemli olan maliyet: **blok okumaları**, karşılaştırmalar değil

Benzetme — sırasız bir kutu rafını okumak

- **Sırasız** bir kutu rafında bir dosya arıyorsunuz
- Bulana kadar **her** kutuyu tek tek açmalısınız
- Ya da tüm kutuları açıp hiçbir şey bulamazsınız — yine hepsini açtınız
- Buradaki bir "kutuyu" bir blok; onu açmak bir disk okuması

Sıralı arama, adım adım



hedef = 67

block reads = 2

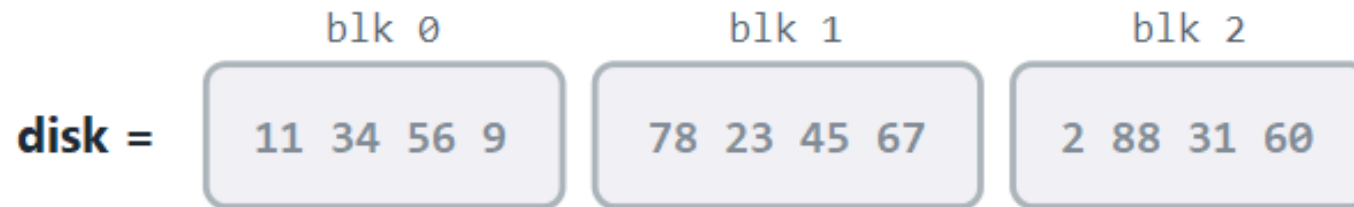
comparisons = 7

bulundu, konum 6



Bitti: 67 konumda 6 bulundu — 2 blok okuması (toplam 4 bloktan), 7 karşılaştırma. Ortalama durumda beklenen okuma sayısı $\approx \text{blok_sayısı}/2$; en kötü durumda tüm bloklar.

Uç durum — hedef yok, dosyanın tamamı taranır



hedef = 999

block reads = 3

comparisons = 12

bulunamadı



Bitti: 999 bulunamadı — bütün dosya tarandı: 3 blok okuması, 12 karşılaştırma. Bulunamama her zaman en kötü durumdur.

Uç durum — hedef ilk kayıt (en iyi durum)



hedef = 70

block reads = 1

comparisons = 1

bulundu, konum 0



Bitti: 70 konumda 0 bulundu — 1 blok okuması (toplam 3 bloktan), 1 karşılaştırma. Ortalama durumda beklenen okuma sayısı $\approx \text{blok_sayısı}/2$; en kötü durumda tüm bloklar.

Kod — bir bloğu tara, sonra bir sonrakini

```
for (int b = 0; b < nblocks; b++) {
    load_block(f, b, buf, &cnt);
    (*block_reads)++;
    for (int i = 0; i < cnt; i++) {
        (*comparisons)++;
        if (buf[i] == target)
            return b * bf + i;
    }
}
return -1; /* exhausted every block */
```

Karmaşıklık

- En kötü durum: $\text{ceil}(n / bf)$ blok okuma — her blok
- Ortalama durum: bunun yaklaşık **yarısı**
- Hafta 1'in $O(n)$ doğrusal aramasının tam dosya analogu
- Sayılan birim **bloklar**, kayıtlar değil — $O(n / bf)$

Yaygın hatalar

- Gerçek maliyet olarak blok okuma yerine **karşılaştırma** saymak
- **Kalan her bloğu** okumadan bir anahtarın yok olduğunu varsaymak
- Yinelenen bir anahtarın **ilk** eşleşmesi yerine sonuncusunu döndürmek

Mini soru

Sıralı aramanın en kötü durumu, hedef yok olsun ya da tam olarak son kayıt olsun, neden tam olarak aynıdır?

Yanıt

İkisi de her bloğun okunmasını gerektirir.

Son kayıt için eşleşme son bloğa kadar bulunmaz; yok olan bir anahtar için hiçbir blok erken elenmez — hepsi kontrol edilmeli.

5. Sıralı Bir Dosyada İkili Arama

Başlangıç sorusu

Bölüm 4'ün dosyası **sıralı** tutulursa, Hafta 1'in ikili aramasının sıralı bir diziyi araması gibi ama blok blok aranabilir mi?

Fikir: BLOKLAR üzerinde ikili arama

- Blok **b** 'nin kayıtları sıralı, örtüşmeyen bir **aralık**
- Hedefi bir bloğun **ilk** ve **son** anahtarıyla karşılaştır
- Küçükse: blok aralığının sağ yarısını at
- Büyükse: sol yarısını at — yoksa hedef bu bloğun **içinde**

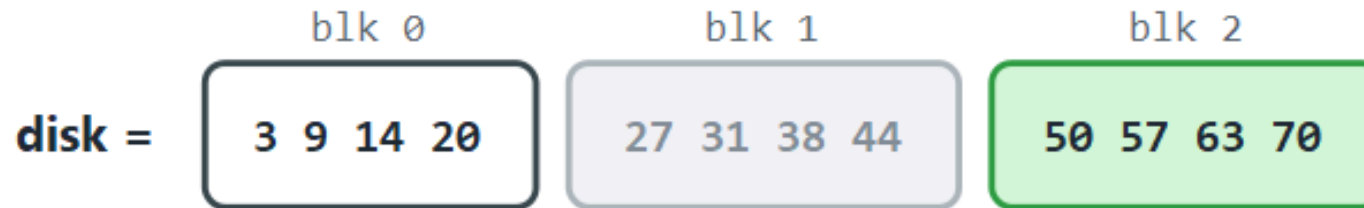
"Boşluk" ne demek

- "Aralıkta" olmak "var" olmak **anlamına gelmez**
- Bir bloğun ilk ve son anahtarı arasındaki bir hedef yine de **yok** olabilir
- O bir bloğun içindeki kısa tarama bunu doğrular ya da yalanlar
- Bu bölüm 4'te asla olmaz — sırasız bir dosyanın aralığı yok

Benzetme — etiketli bir sıralı kutu rafı

- Şimdi rafın kutuları içeriklerinin aralığıyla **etiketli**
- Bir kutunun etiketine bakın: dosyanız ondan önce, içinde, ya da sonra
- İçeremeyeceği her kutuyu doğrudan atlayın
- Yalnız etiket aralığı eşleşen **tek** kutuyu açın

Sıralı dosyada ikili arama, adım adım



hedef = 63

block reads = 2

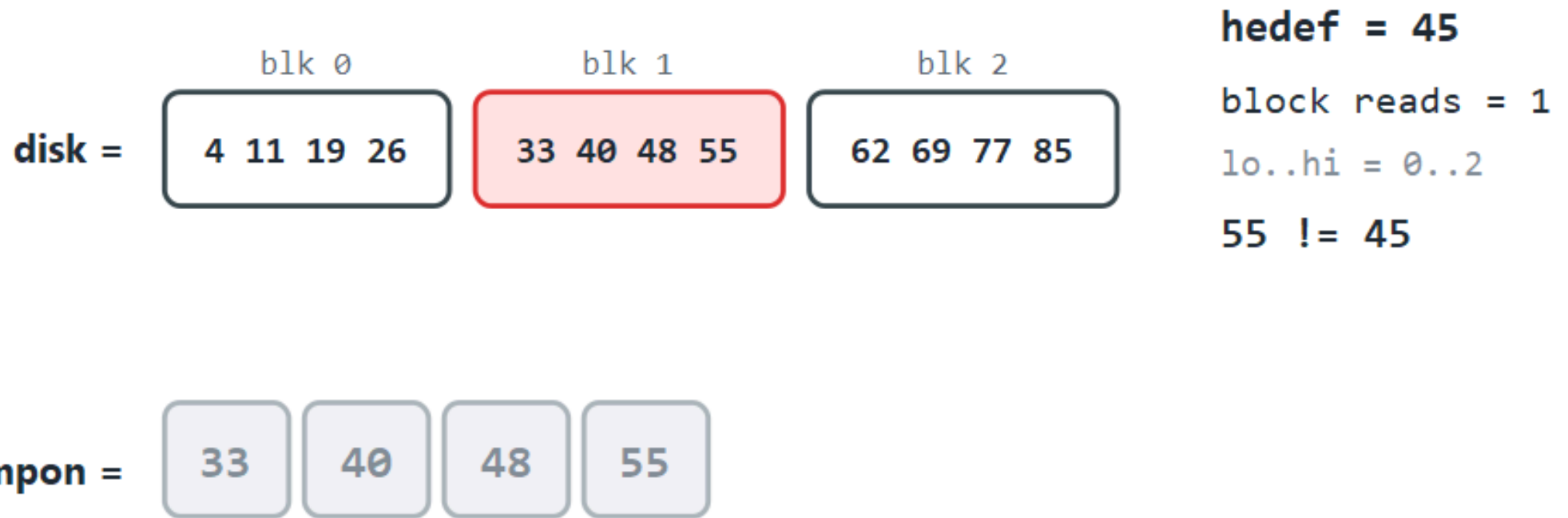
lo..hi = 2..2

63 == 63



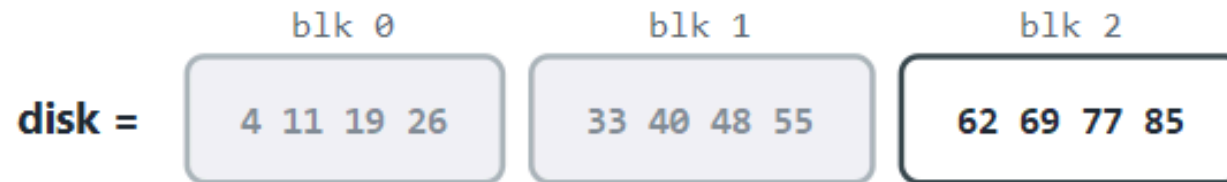
Bitti: 63 konumda 10 bulundu — yalnız 2 blok okuması (3 bloktan; ardışık aramada en kötü durumda 3 okuma gerekirdi). $O(\log \text{ numBlocks}) \approx 2$.

Uç durum — bir bloğun aralığında bir boşluk



Bitti: 45 bulunamadı — 1 blok okuması ile karar verildi (ardışık arama 3 okuma gerektirirdi).

Uç durum — hedef dosyanın en küçük anahtarının altında



hedef = 1

block reads = 2

lo..hi = 0..-1

bulunamadı (aralık dışı)



Bitti: 1 bulunamadı — 2 blok okuması ile karar verildi (ardışık arama 3 okuma gerektirirdi).

Kod — bir bloğa daral, sonra tara

```
while (lo <= hi) {  
    int mid = (lo + hi) / 2;  
    block_reads++;  
    if (target < first_key(mid)) hi = mid-1;  
    else if (target > last_key(mid)) lo = mid+1;  
    else {  
        /* target's range: short scan inside */  
        return scan_block(mid, target);  
    }  
}
```

Karmaşıklık

- $O(\log_2 \text{ numBlocks})$ blok okuma, artı en çok `bf` kayıtlık bir tarama
- Bölüm 4'ün $O(\text{numBlocks})$ 'una karşılaştırın
- 24 anahtar, 6 blok: burada en çok **3** okuma, orada en çok **6**
- Dosya büyüdükçe fark çarpıcı biçimde açılır

Yaygın hatalar

- "Aralıkta" olmanın "var" olmak anlamına gelmediğini unutmak (boşluk durumu)
- Bloğun **orta** anahtarıyla karşılaştırmak (ilk/son yerine)
- Bunu **sırasız** bir dosyaya uygulamak — sessizce yanlış, yalnız yavaş değil

Mini soru

Dizide ikili arama (Hafta 1) $O(\log n)$
karşılaştırma ister. Dosyada ikili arama
 $O(\log \text{ numBlocks})$ okuma ister. Biri diğerinden iyi mi?

Yanıt

Hiçbiri — aynı fikir, farklı birim. Dizi araması *elemanları* yarıya böler; dosya araması *blokları* yarıya böler, ve `numBlocks` zaten `n`'den `bf` kat küçüktür.

6. Sıralı Güncelleme: Ana ve İşlem Dosyasını Birleştirme

Başlangıç sorusu

Bir bankanın hesap dosyası her gün değişir.
Her değişiklik için **tüm** dosyayı yeniden
yazmak milyonlarca kayıta saçma. Alternatif nedir?

Kısa bir tarihçe

- **Sıralı ana + işlem dosyası** birleştirmesi, rastgele erişimli disklerden bile eski, klasik bir toplu işleme kalıbı
- Tharp'ın *File Organization and Processing*'i bunu sıralı dosya güncellemenin kanonik algoritması olarak ele alır
- Bugün hâlâ koca, periyodik toplu işler için kullanılır (fatura kesme)

Fikir: iki SIRALI dosyayı tek geçişte birleştir

- **İşlem dosyası:** sıralı, aynı anahtar, her biri bir **op**'la
- **'A'** ekle, **'C'** değiştir, **'D'** sil
- İki "okuma başı", dosya başına bir — tam olarak Hafta 10'un birleştirme adımı
- Her adımda **master.key** 'i **txn.key** 'le karşılaştır

Üç durum

Karşılaştırma	Anlam	Eylem
<code>master.key < txn.key</code>	Bu kayıt için işlem yok	Değişmeden kopyala
<code>master.key > txn.key</code>	Anahtar henüz anada değil	'A' : ekle; yoksa hata
<code>master.key == txn.key</code>	İşlem burada uygulanır	Sonraki slayta bakın

Eşit-anahtar durumu, ayrıntılı

- Var olan bir anahtara 'A' : yinelenen-ekleme hatası
- Orijinal kayıt yine de **değişmeden kopyalanır**
- 'C' : değer değiştirilir, güncellenmiş kayıt yazılır
- 'D' : kayıt basitçe **yazılmaz** — silme budur

Benzetme — iki sıralı listeyi okuyan iki kişi

- Biri ana listeyi sırayla, yüksek sesle okur
- İkinci kişi işlem listesini sırayla, yüksek sesle okur
- Her adımda şu anki sözcüklerini yüksek sesle karşılaştırırlar
- Alfabetik olarak "geride" olan bir sonraki sözcüğünü okur

Sıralı güncelleme, adım adım

ana dosya =



written = 12

deleted = 1

errors = 0

kalan işlem: ekle

işlemler =



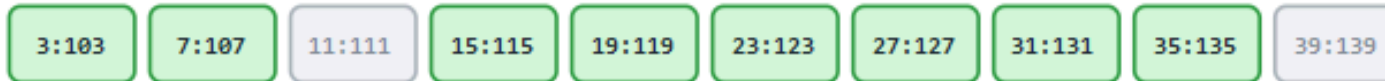
yeni ana dosya =



Bitti: yeni ana dosyada 12 kayıt yazıldı, 1 kayıt silindi, 0 hata. Tek geçişte, iki sıralı dosya paralel taranarak birleştirildi — $O(nm + nt)$.

Uç durum — yinelenen ekleme + yok olan anahtar

ana dosya =



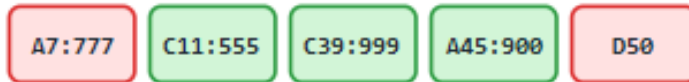
written = 11

deleted = 0

errors = 2

kalan işlem: hata

işlemler =



yeni ana dosya =



Bitti: yeni ana dosyada 11 kayıt yazıldı, 0 kayıt silindi, 2 hata. Tek geçişte, iki sıralı dosya paralel taranarak birleştirildi — $O(nm + nt)$.

Uç durum — ana dosya biter, kalan işlemler

ana dosya =



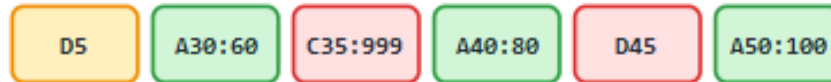
written = 13

deleted = 0

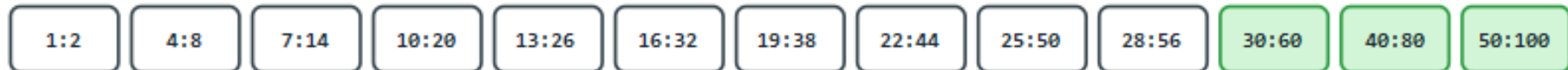
errors = 3

kalan işlem: ekle

işlemler =



yeni ana dosya =



Bitti: yeni ana dosyada 13 kayıt yazıldı, 0 kayıt silindi, 3 hata. Tek geçişte, iki sıralı dosya paralel taranarak birleştirildi — $O(nm + nt)$.

Kod — birleştirme döngüsünün ana karşılaştırması

```
while (i < nm && j < nt) {  
    if (master[i].key < txn[j].key) {  
        write(master[i]); i++;  
    } else if (master[i].key > txn[j].key) {  
        if (txn[j].op == 'A') write(new);  
        else errors++;      /* C/D: missing */  
        j++;  
    } else { /* equal key: next slide */ }  
}
```

Kod — eşit-anahtar dalı, ve kalanlar

```
else {  
    if (txn[j].op == 'A') {  
        write(master[i]); /* KEEP it */  
        errors++;  
    } else if (txn[j].op == 'C') write(new);  
    else deleted++; /* 'D': skip write */  
    i++; j++;  
}  
/* + two leftover loops when one file ends */
```

Karmaşıklık

- $O(nm + nt)$ — tek geçiş, her kayıt tam olarak bir kez okunur
- İşlem başına anayı bir kez aramaya karşılaştırın: $O(nt * nm)$
- Bölüm 4 ya da 5'in araması, nt kez uygulansa, tam bunu maliyet ederdi
- Birleştirme, herhangi gerçek ölçekte çarpıcı biçimde daha ucuzdur

Yaygın hatalar

- Yinelenen-ekleme hatasında ana kaydı **sessizce düşürmek**
- Ana birleştirme bittikten sonra **iki kalan döngüyü** unutmak
- Her iki dosyanın da sıralı olduğunu, kontrol etmeden ya da garanti etmeden varsaymak

Mini soru

Bu algoritmada bir silme neden özel bir "silindi" kaydı yazarak değil, kaydı **yazmayarak** yapılır?

Yanıt

Yeni dosya baştan, kayıt kayıt inşa edilir.

Silinmiş bir kayıt onda hiç bulunmamalı — yazmayı atlamak silmenin *kendisidir*, ek bir muhasebeye gerek yoktur.

7. Göreli (Doğrudan) Dosya Erişimi

Başlangıç sorusu

Bir program zaten "7 numaralı kaydı" istediğini biliyor. Bölüm 4 ve 5 ikisi de *arar*. 7 numaralı kaydı bulmak arama gerektirir mi?

Fikir: hesapla, arama

- Her kayıt bir **görelî kayıt numarası (RRN)** alır: `0, 1, 2, ...`
- `block = rrn / bf`, `offset = rrn % bf`
- Bir bölme, bir mod — **hiç karşılaştırma yok**
- Bir `fseek`, bir blok okuma — gerçek **$O(1)$**

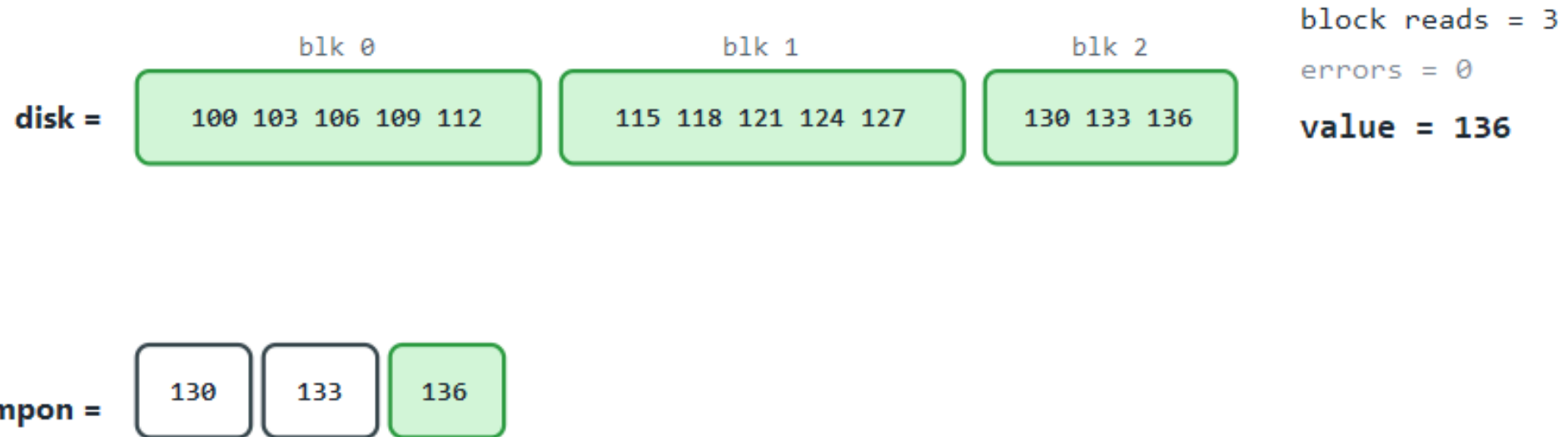
Hâlâ doğrulanması gereken

- Negatif `rrn`, ya da `rrn >= total` : herhangi bir okumadan **önce** reddet
- Geçerli görünen bir `rrn` yine de **kısmi son bloğa** düşebilir
- Yalnız blok sınırlarına değil, **toplam kayıt sayısına** karşı kontrol gerekir

Benzetme — numaralı bir otopark katı

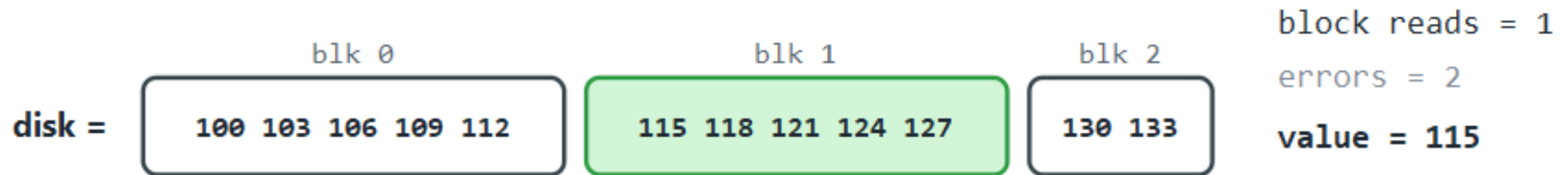
- Her park yerinin zeminde boyalı bir **numarası** var
- "37 numaralı yer" arama gerektirmez — doğrudan 37/bf katına git
- Bir otopark görevlisi 37 numaralı yeri her katı gezerek aramaz
- Tam olarak bölüm 7'nin $\text{block} = \text{rrn} / \text{bf}$, $\text{offset} = \text{rrn} \% \text{bf}$ 'i

Görelî dosya erişimi, adım adım



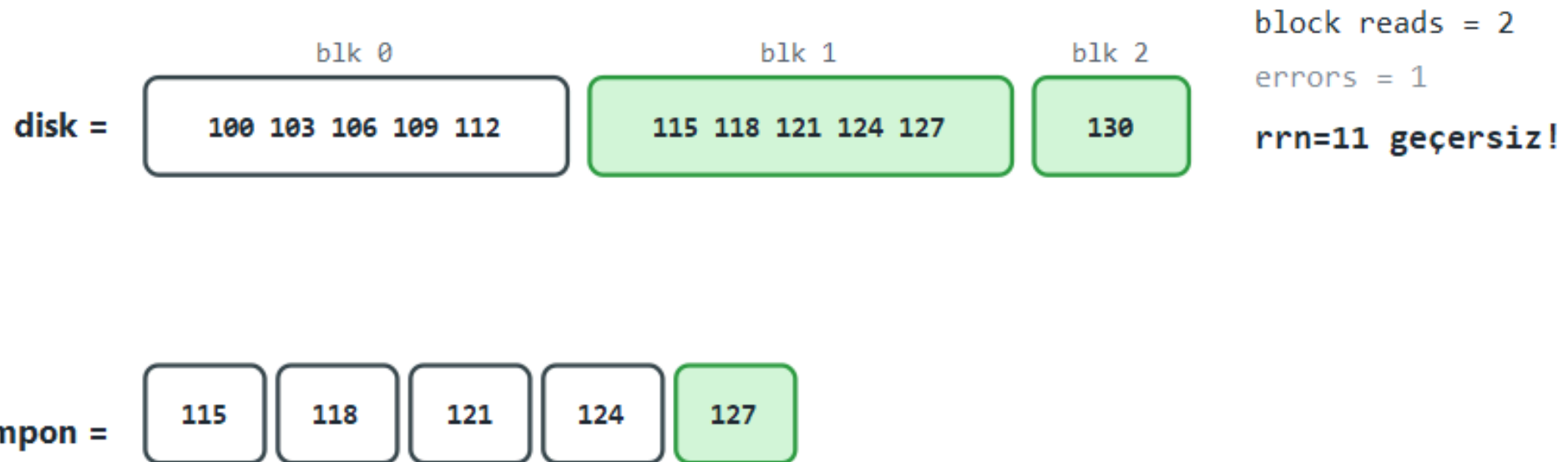
Bitti: 3 istek, 3 blok okuması (her geçerli istek tam 1 okuma), 0 hata. Ardışık ya da ikili aramanın aksine, görelî erişimde okuma sayısı dosya boyutundan bağımsızdır.

Uç durum — negatif ve aralık dışı istekler



Bitti: 3 istek, 1 blok okuması (her geçerli istek tam 1 okuma), 2 hata. Ardışık ya da ikili aramanın aksine, görelî erişimde okuma sayısı dosya boyutundan bağımsızdır.

Uç durum — kısmi son bloktaki tek kayıt



Bitti: 3 istek, 2 blok okuması (her geçerli istek tam 1 okuma), 1 hata. Ardışık ya da ikili aramanın aksine, görelî erişimde okuma sayısı dosya boyutundan bağımsızdır.

Kod — tüm algoritma, iki satırda

```
int direct_read(FILE *f, int bf,
                int total, int rrn) {
    if (rrn < 0 || rrn >= total)
        return -1;          /* reject */
    int block = rrn / bf;
    int offset = rrn % bf;
    /* one fseek, one fread, done */
}
```

Karmaşıklık

- $O(1)$ — her geçerli istek için tam olarak bir blok okuma
- Bölüm 4 ve 5'in maliyetleri dosya büyüdükçe **büyümeye devam eder**
- Doğrudan erişim hiç büyümmez, asla
- Bu haftanın geri kalan tekniklerinin yaklaştırmaya çalıştığı tavan

Yaygın hatalar

- `rrn >= total` 'i kontrol edip `rrn < 0` 'i unutmak
- **Kısmi son bloğu** hesaba katmamak
- "Doğrudan"ın "doğrulama gerekmez" anlamına geldiğini varsaymak — yine gerekir

Mini soru

Görelî erişimin $O(1)$ maliyeti neden hiç `bf` 'e bağılı değilken, bölüm 4 ve 5'in arama maliyetlerinin ikisi de bağılıdır?

Yanıt

Doğrudan erişim, **bf** 'ten bağımsız olarak
her zaman tam bir seek ve bir okuma yapar
— yalnız *hangi* blok değişir. Arama ise
cevabı bulana kadar blokları tek tek ziyaret etmelidir.

8. Kovalara Hash'leme

Başlangıç sorusu

Gerçek anahtarlar nadiren uygun bir `0, 1, 2, ...`.

Hafta 6 bunu RAM'de bir hash fonksiyonuyla
çözdü. Aynı fikir bir dosya için çalışır mı?

Fikir: bir KOVAYA hash'le, taşmada zincirle

- $h(\text{key}) = \text{key} \bmod m$, m **kovadan** birini seçer
- Bir kova bir **bloktur**, bf 'e kadar anahtar tutar — bir hücre değil
- Aynı kovayı paylaşan iki anahtar sorun değil, **kova dolana kadar**
- O zaman: bir **taşma bloğu** ayrılır, dolu kovaya zincirlenir

Benzetme — birkaç mektup tutan posta kutuları

- Her posta kutusu (kova) zaten `bf` 'e kadar mektup (anahtar) tutar
- Aynı kutuya iki mektup: sorun değil, kutu dolana kadar
- Dolu bir posta kutusuna **ekstra bir tepsi** takılır
- O ekstra tepsi tam olarak zincirlenen bir taşma bloğu

Kovalara hash'leme, adım adım



taşma =

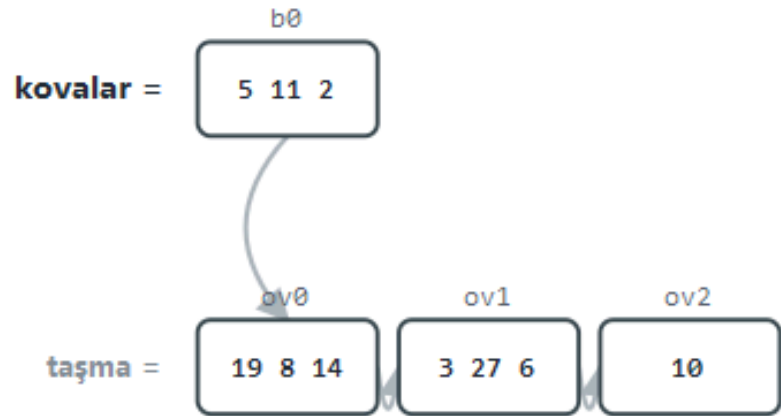
RAM tampon =



writes = 13
blocks used = 5
 $h(21)=21\%5=1$

Bitti: 13 anahtar, 5 ana kova, 0 taşma bloğu (toplam 5 blok), 13 yazma. Yükleme çarpanı = $13 / (5 \times 3) = 0.87$.

Uç durum — $m=1$, her şey zincirlenir



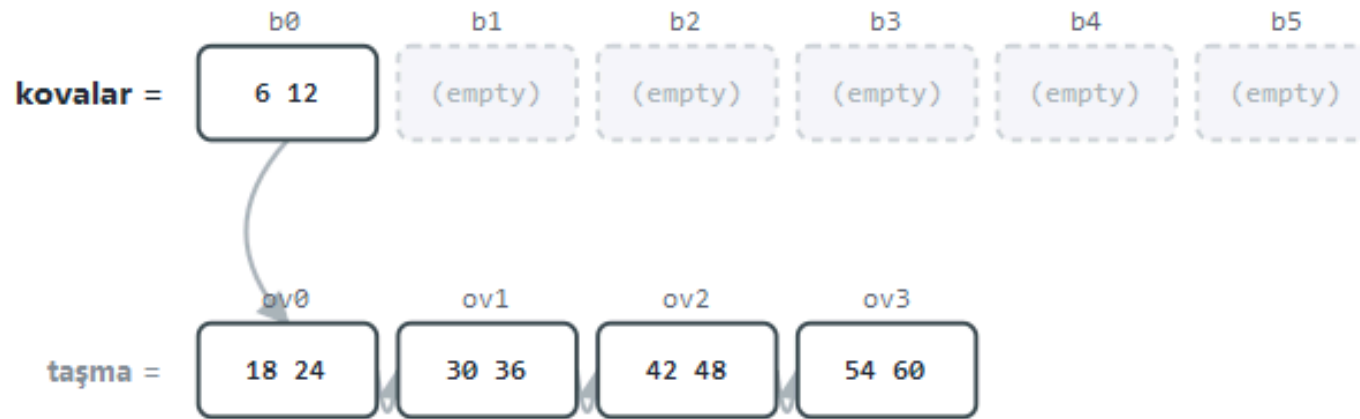
RAM tampon =

10

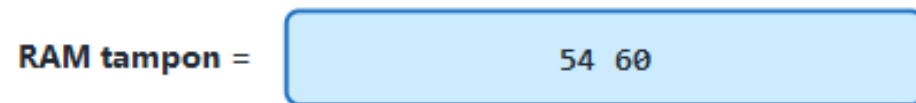
writes = 13
blocks used = 4
 $h(10)=10\%1=0$

Bitti: 10 anahtar, 1 ana kova, 3 taşma bloğu (toplam 4 blok), 13 yazma. Yükleme çarpanı = $10 / (1 \times 3) = 3.33$.

Uç durum — 10 anahtarın tümü aynı kovaya



writes = 14
blocks used = 10
 $h(60) = 60 \% 6 = 0$



Bitti: 10 anahtar, 6 ana kova, 4 taşma bloğu (toplam 10 blok), 14 yazma. Yükleme çarpanı = $10 / (6 \times 2) = 0.83$.

Kod — hash'le, taşmada zincirle

```
int h = key % m, cur = h;
while (bucket[cur].count == bf) {
    if (bucket[cur].next < 0)
        bucket[cur].next = new_block();
    cur = bucket[cur].next;    /* follow chain */
}
bucket[cur].keys[bucket[cur].count++] = key;
```

Karmaşıklık

- Ortalama durum: **$O(1)$** blok erişimi, Hafta 6'nın zincirlemesiyle aynı
- Artı, izlenmesi gereken her taşma bloğu için bir erişim daha
- **Yük faktörü** ($n / (m * bf)$) makul olduğu sürece küçük kalır
- En kötü durum ("m=1" yukarıda): $O(n / bf)$ 'ye doğru bozulur

Yaygın hatalar

- Yeni taşma bloğunu **yanlış** bloğa zincirlemek
- Beklenen anahtar sayısı için **m** 'i çok küçük seçmek
- Bir kovanın kapasitesinin **bf** olduğunu, 1 olmadığını unutmak

Mini soru

Taşma bloklarını burada zincirlemek, Hafta 6'nın ayrık zincirlemesine mi, yoksa açık adreslemesine mi daha yakın? Neden?

Yanıt

Ayrık zincirleme. Dolu bir kova, kenarda **ek bloklardan** bir zincir büyütür — başka hiçbir kovaya dokunmadan, tam olarak Hafta 6'nın hücre başına bağlı listesi gibi.

9. İlerleyici Taşma: Diskte Doğrusal Yoklama

Başlangıç sorusu

Bölüm 8'in taşma blokları, her taşan kova için tam bir ekstra bloğa mal olur. Yoklama bu israfı tamamen önleyebilir mi?

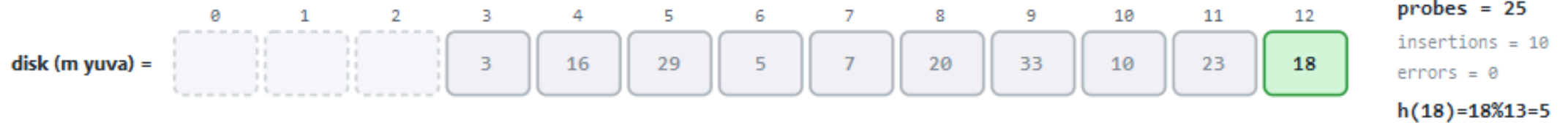
Fikir: bir sonraki hücreyi yokla, sarmalayarak

- Ana hücre: $h(\text{key}) = \text{key} \bmod m$
- Farklı bir anahtarla dolu mu? **Sonraki** hücreyi dene, $(h+1) \bmod m$
- $\bmod m$, diziyi son hücreden 0'a **sarmalar**
- Dur: boş hücre (ekle), yinelenen, ya da m hücrenin tümü denendi (dolu)

Benzetme — en yakın boş yere park etmek

- Size ayrılan yer dolu mu? Bir sonraki yeri alın
- O da dolu mu? Bir sonraki — sonda yer 0'a sarmalayılarak
- Hiç ayrı bir taşma alanı yok — herkes aynı alana park eder
- Tam olarak Hafta 6'nın açık adreslemesi, diskte bir kademe aşağıda

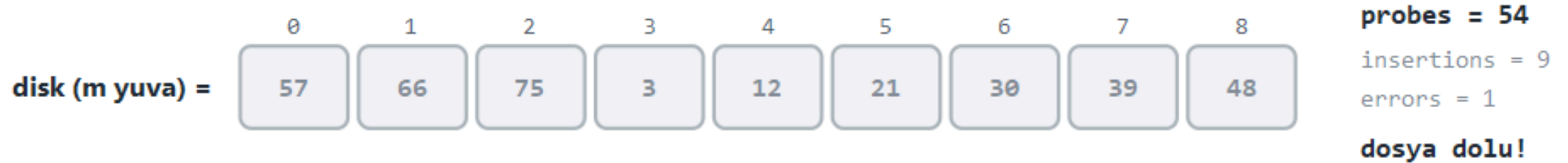
İlerleyici taşıma, adım adım



RAM: yoklanan =

Bitti: 10 istek, 10 eklendi, 0 hata (yinelenen/dosya dolu), toplam 25 yoklama. Yoklama zinciri uzadıkça (asal kümelenme) sonraki eklemeler de yavaşlar.

Uç durum — her anahtar bir hücreyi paylaşıyor



RAM: yoklanan =

Bitti: 10 istek, 9 eklendi, 1 hata (yinelenen/dosya dolu), toplam 54 yoklama. Yoklama zinciri uzadıkça (asal kümelenme) sonraki eklemeler de yavaşlar.

Uç durum — tablo tamamen dolu



RAM: yoklanan =

Bitti: 10 istek, 9 eklendi, 1 hata (yinelenen/dosya dolu), toplam 18 yoklama. Yoklama zinciri uzadıkça (asal kümelenme) sonraki eklemeler de yavaşlar.

Kod — yer, yinelenen ya da dolu bulana kadar yokla

```
int h = key % m, i = h, tries = 0;
while (tries < m) {
    if (table[i] == EMPTY) {
        table[i] = key; return i;
    }
    if (table[i] == key) return -2; /* dup */
    i = (i + 1) % m; tries++;
}
return -1; /* file genuinely full */
```

Karmaşıklık

- **Düşük** yük faktöründe n / m , $O(1)$ 'e yakın yoklama
- Yük faktörü yükseldikçe keskin biçimde bozulur
- "Hard" senaryo (%91 dolu): bir ekleme için 5 yoklamaya kadar
- Nerdeyse tam dolu en kötü durum: **$O(m)$** yoklama

Yaygın hatalar

- Yoklama döngüsünü sınırlamamak (`tries < m`)
- "Dosya dolu"yu çökme yerine normal bir sonuç olarak ele almamak
- `key % m` için negatif anahtar korumasını unutmak

Mini soru

"Birincil kümelenme" nedir, ve "aynı ana hücreyi paylaşan her anahtar" uç durumu bunu neden en uç haliyle gösterir?

Yanıt

Daha fazla çakışma çeken büyüyen dolu bir seri. Burada 10 anahtarın tümü aynı ana hücreyi paylaşıyor, yoklama sayıları kilitlenmiş biçimde **1, 2, 3, ...** artıyor.

10. Mezar Taşlarıyla Silme

Başlangıç sorusu

Silinen bir hücre basitçe `EMPTY` 'ye temizlenirse, o hücreden geçmek zorunda kalmış farklı bir anahtarın sonraki aramasına ne olur?

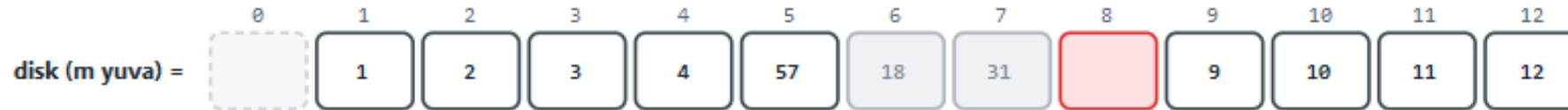
Fikir: bir mezar taşı "aramaya devam et" demektir

- **Mezar taşı**: hem `EMPTY` 'den hem gerçek bir anahtardan farklı
- Bir arama bir mezar taşı, herhangi bir dolu hücre gibi **atlar**
- Yalnız gerçek bir `EMPTY` hücre bir aramayı **bitirebilir**
- Bir **ekleme** ise bir mezar taşı, yeniden kullanmakta özgürdür

Benzetme — "gitti, ama burada durma" tabelası

- Bir park yerinin sahibi ayrıldı: geçici bir **tabela** dikin
- Tabela "boş" (dolaşmaya devam et) ya da "dolu" (girme) demez
- Birinin bir zamanlar burada park ettiğini, arabanızın daha ilerde olabileceğini söyler
- Yeni bir araba yine de buraya park edebilir — o zaman tabela iner

Mezar taşlarıyla silme, adım adım

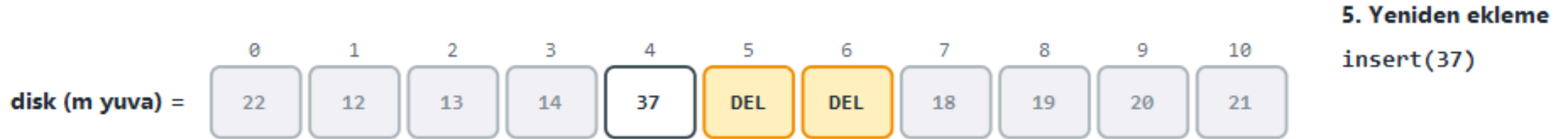


5. Yeniden ekleme
insert(57)

RAM: yoklanan =

Bitti: mezar taşı, silinen bir yuvayı "boş" değil "burada bir şey vardı, aramaya devam et" olarak işaretler — arama doğruluğunu korur, ekleme ise onu yeniden kullanarak yer kazanır.

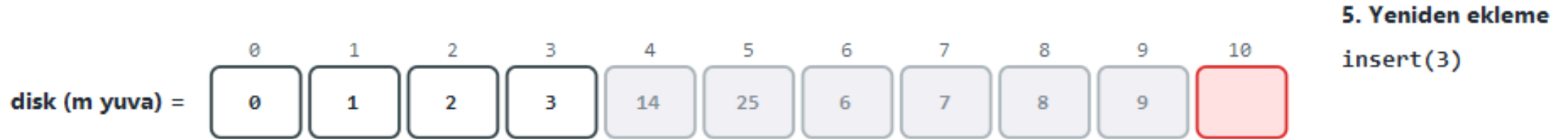
Uç durum — tamamen mezar taşlanmış bir tablo



RAM: yoklanan =

Bitti: mezar taşı, silinen bir yuvayı "boş" değil "burada bir şey vardı, aramaya devam et" olarak işaretler — arama doğruluğunu korur, ekleme ise onu yeniden kullanarak yer kazanır.

Uç durum — bir anahtarı sil, sonra aynı anahtarı yeniden ekle



RAM: yoklanan =

Bitti: mezar taşı, silinen bir yuvayı "boş" değil "burada bir şey vardı, aramaya devam et" olarak işaretler — arama doğruluğunu korur, ekleme ise onu yeniden kullanarak yer kazanır.

Kod — find mezar taşlarını atlamalı, durmamalı

```
int i = key % m, tries = 0;
while (tries < m) {
    if (table[i] == EMPTY) return -1;
    if (table[i] == key) return i;
    /* TOMB: fall through, keep probing */
    i = (i + 1) % m; tries++;
}
return -1;
```

Kod — ekleme bir mezar taşını yeniden kullanabilir

```
int i = key % m, tries = 0;
while (tries < m) {
    if (table[i] == EMPTY ||
        table[i] == TOMB) {
        table[i] = key; return i; /* reused! */
    }
    if (table[i] == key) return -2;
    i = (i + 1) % m; tries++;
}
```

Karmaşıklık

- `ts_find` / `ts_delete` : bölüm 9'un düz yoklamasıyla aynı **$O(\text{yoklama zinciri uzunluğu})$**
- Mezar taşları hiçbir yeni asimptotik maliyet eklemmez
- `ts_insert` : pratikte genelde **daha az** — erken bir mezar taşını yeniden kullanabilir
- Yalnız bir aramanın hangi hücrelerden geçmeye razı olduğu değişir

Yaygın hatalar

- Silerken hücreyi basitçe `EMPTY` 'ye temizlemek
- `ts_find` 'ın bir mezar taşında durmasını ya da onu eşleşme saymasını sağlamak
- Tamamen mezar taşlanmış bir tablonun güvenilecek **hiç** `EMPTY` 'si olmadığını unutmak

Mini soru

`ts_insert` bir mezar taşını yeni bir anahtar için yeniden kullandıktan sonra, o hücreden geçen ilgisiz bir anahtarın araması neden hâlâ doğru çalışır?

Yanıt

Yeniden kullanıldıktan sonra hücre yine gerçek bir anahtar tutar — başka herhangi bir dolu hücreden ayırt edilemez. Diğer anahtarların aramaları hiç değişmemiş gibi davranır.

11–12. Karşılaştırma ve Seçim

Karşılaştırma tablosu

Organizasyon	Bulma	Ekleme / Silme	En iyi olduğu yer
Sıralı (sırasız)	$O(\text{numBlocks})$	$O(1) / O(\text{numBlocks})$	Zaten her kaydı işlemek
Sıralı (sıralı)	$O(\log \text{ numBlocks})$	$O(\text{numBlocks})$ ikisi de	Toplu güncellemeler (bölüm 6)
Görelî (doğrudan)	$O(1)$	$O(1) / O(1)$, boşluk bırakır	RRN doğal olarak biliniyor
Hash'leme (kova/yoklama)	$O(1)$ ort.	$O(1)$ ort.	Keyfi anahtar, hızlı arama

Teknik seçimi (1/2)

Durum	En iyi seçim
Zaten her kaydı işleyeceksiniz	Sıralı, herhangi bir sırada
Periyodik toplu değişiklikler	Sıralı (sıralı) + işlem dosyası
Ara sıra arama, nadiren değişir	Sıralı (sıralı), ikili arama
Kayıtlar 0, 1, 2, ... numaralı	Görelî (doğrudan) dosya

Teknik seçimi (2/2)

Durum	En iyi seçim
Keyfi anahtar, sadelik öncelikli	Hash'leme, kova zincirleme
Keyfi anahtar, kompaktlık, nadir silme	Hash'leme, ilerleyici taşıma
Keyfi anahtar, sık silme	İlerleyici taşıma + mezar taşları

Kapanış

Özet

- **Kayıt/alanlar:** sabit, sınırlandırılmış, ya da uzunluk önekli
- **Bloklama çarpanı:** `bf` kayıt bir disk erişimini paylaşır
- **Sıralı:** her bloğu sırayla oku, ya da sıralıysa blok düzeyinde ikili ara
- **Sıralı güncelleme:** ana + işlemleri tek geçişte birleştir
- **Görelî erişim:** `block = rrn/bf` — gerçek $O(1)$
- **Hash'leme:** taşma bloklarını zincirle, ya da mezar taşlarıyla yokla

1. Bu hafta neden karşılaştırma değil blok okuma sayıyor?

Bir disk erişimi herhangi bir RAM işleminden çok daha pahalıdır, ve her zaman TAM bir blok aktarır — bu yüzden karşılaştırma değil blok sayısı gerçek hızı öngörür.

2. Sıralı bir dosyada ikili arama neden orta anahtar yerine bir bloğun ilk/son anahtarıyla karşılaştırmalı?

Her karşılaştırmada koca bir BLOK kayıt içeri/dışarı bırakılıyor, ve yalnız ilk/son anahtar o bloğun aralığını tanımlar.

3. Yinelenen-ekleme hatası neden yine de orijinal ana kaydı ileri kopyalamalı?

Ana kayıt hiçbir yanlışı yapmadı — yalnız işlem reddedilir; onu ileri kopyalamamak sessizce geçerli veriyi kaybederdi.

4. Bir ekleme neden bir mezar taşını güvenle yeniden kullanabilirken, bir arama onda durmamalıdır?

Arama diğer HER anahtarın bulunabilme yeteneğini korumalıdır; eklemenin yeniden kullanımı buna müdahale etmez, çünkü hücre sonra yeni, gerçek bir anahtar tutar.

5. Göreli bir dosyanın $O(1)$ maliyeti neden dosyanın toplam boyutuna hiç bağlı değil?

rrn ve bf'ten hesaplanan tek bir fseek, sonra bir blok okuma —
bu hesaplama, dosyanın BAŞKA kaç kayıt tuttuğuna hiç bağlı
değildir.

6. Taşma bloklarını zincirlemek (bölüm 8) neden ayrık zincirlemeye açık adreslemeden daha yakın?

Dolu bir kova kenarda EK bloklardan bir zincir büyütür, başka hiçbir kovaya dokunmadan — tam olarak Hafta 6'nın ayırık zincirlemesi, bir kademe aşağıda.

Gelecek hafta

Hafta 14 — Dosya Organizasyonu II

İndekslenmiş sıralı dosyalar · B-ağaçları ·
genişleyebilir hash'leme · dış sıralama — bu
haftanın blok-okuma maliyet modelini daha büyük araçlara genişletiyor.

Kaynaklar (1/2)

- Ders izlencesi, Hafta 13: `docs/syllabus/syllabus.en.md`
- A. L. Tharp. *File Organization and Processing*.
John Wiley & Sons, 1988 — birincil ders kitabı
- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*. MIT Press — hash tablosu bölümü

Kaynaklar (2/2)

- Knuth. *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2. baskı — dosya işleme, hash'leme ve doğrusal yoklamanın tarihsel analizi
- [williamfiset/Algorithms](#) · Programiz DSA